

Polimorfizm statyczny i dynamiczny

Piotr Janowski

Polimorfizm

Polimorfizm (z greckiego "wiele form") to jeden z filarów programowania obiektowego. Pozwala on na traktowanie obiektów różnych klas w jednolity sposób. Dzięki polimorfizmowi, można używać metody o tej samej nazwie na obiektach różnych typów, a każdy z nich wykona ją w sposób unikalny dla swojej klasy.

W C# polimorfizm osiąga się na dwa główne sposoby:

- 1. Polimorfizm statyczny (przeciążanie metod):** Polega na tworzeniu kilku metod o tej samej nazwie, ale z różnymi listami argumentów (różna liczba lub typy parametrów). Kompilator decyduje, która metoda ma zostać wywołana, już na etapie kompilacji.
- 2. Polimorfizm dynamiczny (nadpisywanie metod):** Umożliwia klasie pochodnej zdefiniowanie własnej implementacji metody, która jest już zdefiniowana w klasie bazowej. W tym celu używa się słów kluczowych **virtual** w klasie bazowej i **override** w klasie pochodnej.

Polimorfizm

```
class Calculator
{
    Odwołania: 0
    public int Add(int a, int b) => a + b;
    Odwołania: 0
    public double Add(double a, double b) => a + b; // przeciążenie
}
```

Kluczowe słowa w polimorfizmie dynamicznym

- virtual** – oznacza, że metoda może być nadpisana w klasie pochodnej.
- override** – nadpisuje metodę virtual z klasy bazowej.
- abstract** – wymusza implementację w klasach dziedziczących.
- new** – ukrywa metodę klasy bazowej (tzw. *method hiding*).

```
class Animal
{
    Odwołania: 4
    public virtual void Speak() => Console.WriteLine("Zwierzę wydaje dźwięk");
}

1 odwołanie
class Dog : Animal
{
    Odwołania: 3
    public override void Speak() => Console.WriteLine("Hau hau!");
}

1 odwołanie
class Cat : Animal
{
    Odwołania: 3
    public override void Speak() => Console.WriteLine("Miau!");
}

Odwołania: 0
class Program
{
    Odwołania: 0
    static void Main()
    {
        Animal a1 = new Dog();
        Animal a2 = new Cat();

        a1.Speak(); // Hau hau!
        a2.Speak(); // Miau!
    }
}
```



Polimorfizm

Kluczowe słowa w polimorfizmie dynamicznym

- override** – nadpisuje metodę virtual z klasy bazowej.
- abstract** – wymusza implementację w klasach dziedziczących.

```
using System;

abstract class Shape
{
    public abstract double GetArea(); // metoda bez implementacji
}

class Circle : Shape
{
    public double Radius { get; set; }

    public Circle(double r) => Radius = r;

    public override double GetArea() => Math.PI * Radius * Radius;
}

class Square : Shape
{
    public double Side { get; set; }

    public Square(double s) => Side = s;

    public override double GetArea() => Side * Side;
}

class Program
{
    static void Main()
    {
        Shape s1 = new Circle(5);
        Shape s2 = new Square(4);

        Console.WriteLine($"Koło pole = {s1.GetArea()}");
        Console.WriteLine($"Kwadrat pole = {s2.GetArea()}");
    }
}
```



Polimorfizm

Kluczowe słowa w polimorfizmie dynamicznym

- **new** – ukrywa metodę klasy bazowej (tzw. *method hiding*).

```
using System;

class Animal
{
    public void Speak() => Console.WriteLine("Zwierzę wydaje dźwięk");
}

class Dog : Animal
{
    // Ukrywa metodę klasy bazowej, zamiast ją nadpisywać
    public new void Speak() => Console.WriteLine("Hau hau!");
}

class Program
{
    static void Main()
    {
        Animal a = new Dog();
        Dog d = new Dog();

        a.Speak(); // Wywoła metodę z Animal (Zwierzę wydaje dźwięk)
        d.Speak(); // Wywoła metodę z Dog (Hau hau!)
    }
}
```

Podsumowanie

Polimorfizm statyczny (kompilacyjny)

- › Oparty na **przeciążaniu metod** (*method overloading*) i operatorów.
- › Decyzja, którą wersję metody wywołać, zapada w czasie **kompilacji**.

Polimorfizm dynamiczny (dziedziczenie, interfejsy)

- › Oparty na **nadpisywaniu metod** (*method overriding*) w klasach pochodnych.
- › Decyzja, którą metodę wywołać, zapada w czasie **uruchomienia (runtime)**.

Zalety i wady polimorfizmu

Zalety polimorfizmu

- Zwiększa elastyczność i rozszerzalność kodu,
- Pozwala pisać kod bardziej abstrakcyjny i ogólny,
- Ułatwia utrzymanie i rozszerzanie aplikacji,
- Sprzyja zasadzie otwarte-zamknięte (OCP) z SOLID – kod jest otwarty na rozszerzenia, a zamknięty na modyfikacje.

Wady / pułapki

- Nadużywanie polimorfizmu może komplikować strukturę kodu.
- Wydajność – polimorfizm dynamiczny (wirtualne wywołania) jest minimalnie wolniejszy niż statyczny.
- new (ukrywanie metod) bywa źródłem nieporozumień.

Dobre praktyki

- Używaj polimorfizmu do ekstensji systemu, zamiast kopiować kod.
- Wybieraj interfejsy zamiast rozbudowanej hierarchii klas, gdy zależy Ci na elastyczności.
- Unikaj nadużywania new – preferuj override.
- Testuj polimorficzne zachowania (szczególnie przy skomplikowanych hierarchiach).

Zadania teoretyczne

1. Wyjaśnij różnicę między **polimorfizmem statycznym** a **dynamicznym** w C#.
2. Czym różni się słowo kluczowe **abstract** od **virtual**?
3. Dlaczego w praktyce lepiej używać **override** zamiast **new**?
4. Podaj przykłady polimorfizmu w życiu codziennym (np. różne typy pojazdów, które mają wspólną metodę Jedź()).
5. Jakie są zalety stosowania interfejsów w kontekście polimorfizmu?